NATIONAL UNIVERSITY OF LESOTHO

Department of Mathematics and Computer Science

CS4433 — SOFTWARE ENGINEERING

MODERATED LEARNING MANAGEMENT SYSTEM

Final Software Engineering Report

Student: Itumeleng Raphoto

Student number: 202320764

Programme: BSc Information Systems

Development model: Waterfall with V-Model verification and validation

Submission: Individual assignment — 7 weeks

Date: 7 November 2026

Abstract

This report presents the analysis, design, implementation, verification, quality assurance, deployment and maintenance of a moderated Learning Management System (LMS) for a secondary school. The system supports four roles: System Administrator, Lecturer, Head of Department (HoD) and Student. It manages accounts, departments, courses, enrolments, assignments, extensions, submissions, marking, moderation and release. The implementation is a C++17 console application with SQLite persistence, parameterised data access, OpenSSL PBKDF2 password verification and a persistent mark-change audit trail. The project applies the Waterfall model with V-Model verification gates. The final system passes 39 of 39 automated tests. Business-logic line coverage is 80.7%; all-source coverage, including interactive console code, is reported separately as 59.8%. A post-release change request adds whole-class deadline extensions in version 1.1 without breaking existing behaviour. Detailed diagrams, screenshots, test evidence and user documentation are linked from the GitHub repository rather than embedded in this report, keeping the report readable and within the submission scope.

1. Introduction

The case study is a school LMS that must provide a controlled assessment workflow. The school requires a simple local application because staff may not be IT specialists and the IT officer expects operation on older computers with unreliable internet. The application must also protect marks and personal information. These constraints led to a console-based design, local SQLite persistence and a strict separation between Lecturer marking, HoD moderation and Student result visibility.

The central problem is not merely storing assignments. The system must preserve the history of submissions, calculate late penalties fairly, permit approved extensions, prevent unauthorised access, support resubmission, and provide evidence when marks change. The project therefore treats access control, data integrity and auditability as core requirements rather than optional features.

The scope excludes online payment, video conferencing, automated grading, plagiarism detection and web/mobile interfaces. The intended deployment is a single school computer or a local installation. The report follows the required assignment structure, except that the requested table of contents is deliberately omitted. Figures are referenced as short external links to the repository.

Project repository: CS4433-LMS-Phase4

2. Project management plan

2.1 Feasibility and planning

Technical feasibility was positive because C++17, CMake, SQLite, OpenSSL and GoogleTest are available open-source technologies that support offline operation. The highest technical risks were consistent authorisation, correct lifecycle rules, transaction rollback and secure password handling. SQLite is suitable for a single-host deployment with a moderate number of stored records; the project does not claim that 800 students can connect concurrently to a database file over an unreliable network. That distinction is recorded as an assumption for responsible engineering.

Operational feasibility was positive because each stakeholder has a distinct workflow. Role-labelled menus and plain-language validation messages reduce training needs. Economic feasibility was also positive because the assignment can use free tools and existing hardware, although a school would still need a backup process and a responsible data owner. Legal feasibility is conditional on the school confirming its obligations for educational and child-related personal information under applicable Lesotho data-protection law.

2.2 Schedule, gates and resources

The seven-week schedule follows the assignment phases: planning; requirements; design; implementation; verification; deployment; maintenance and completion. Approximately 70 focused hours were estimated, or about 10 hours per week. The critical path runs from requirements approval to design approval, implementation, verification, version 1.0 deployment, the CR-01 change and final reporting. The plan reduces schedule risk by writing tests and report sections alongside implementation rather than postponing all evidence to the final week.

Gate	Decision evidence
Gate 1	Approved stakeholder decisions, SRS, use cases and traceability.
Gate 2	Layered architecture, UML, database design, security approach and test plan.
Gate 3	Passing automated tests, acceptance walkthroughs, coverage, static analysis and release evidence.

2.3 Risk management

The highest initial exposures were schedule compression and performance uncertainty. Other high risks included unresolved policy rules, stakeholder availability, security defects, data corruption, late testing and usability problems. Mitigations included an assumption log, weekly progress reviews, centralised authorisation, transaction use, database backup instructions, boundary-value tests and a release rule that treats failed security or audit tests as blockers.

Risk	Mitigation and contingency
Requirements conflict	Baseline decisions at Gate 1; trace every rule to a test.
Schedule overrun	Protect weekly hours; defer optional interface polish; retain verification time.
Unauthorised access	Centralise role and record-scope checks; test negative permission cases.
Database loss	Use transactions and constraints; document backup and restore; never commit real databases.
Late testing	Create tests with each feature and run CI throughout implementation.

Detailed planning and risk evidence: Phase 1 project planning document

3. Requirements engineering

3.1 Stakeholder analysis and conflict resolution

The Principal prioritised simplicity and confidentiality of marks. Lecturers required assignment management, extensions, marking and feedback. The HoD required department-wide moderation before release. Students required course visibility, submission receipts, resubmission and written feedback. The IT officer required offline operation, password protection and reasonable performance. The System Administrator required account, department, course, assignment and enrolment administration. The assessor required a verifiable SRS and traceability.

Conflicts were resolved by defining measurable rules. Late submissions are accepted while the assignment remains active, but a 10% of maximum-mark deduction is applied for each late day up to three days; more than 72 hours late receives zero. A student may submit multiple versions before the effective deadline, with the latest on-time version selected for marking. Marks and feedback remain hidden until the complete batch is approved and released by the departmental HoD.

3.2 Software requirements specification

ID	Requirement summary	Verification
FR-01–02	Login, logout and role/record-scope authorisation.	Positive and negative permission tests.
FR-03–05	Account, department, course, lecturer assignment and enrolment administration.	Service tests and acceptance workflow.
FR-06–07	Assignment creation/editing and individual extensions.	Validation and scope tests.

FR-08–10	Student visibility, versioned submissions and late recording.	Deadline and version tests.
FR-11–12	Penalty calculation, marking and feedback.	Boundary-value marking tests.
FR-13–14	HoD moderation/release and student result visibility.	Release-blocking and visibility tests.
NFR-01–06	Performance, offline operation, password confidentiality, authorisation, audit and data integrity.	Evidence, static analysis and fault/rollback tests.

The requirements are measurable: marks are bounded by zero and the assignment maximum; effective deadlines are explicit; unauthorised actions must be denied without data change; and audit persistence must be atomic with mark changes. These measurable definitions make the later tests meaningful rather than relying on the statement that the system "works."

Figure 1. Stakeholder and requirements artefacts, including the SRS and use-case model. View the figure in the GitHub repository

Requirements evidence: SRS and use-case document, requirements traceability matrix

4. Software design document

4.1 Architecture

A layered architecture was selected. The console UI presents menus and collects input. Application services coordinate use cases and enforce business rules. The data-access layer owns SQLite statements and transactions. Domain objects and role classes represent users and workflow concepts. This design prevents the UI from issuing SQL directly, improves testability and provides a clear location for authorisation and validation.

Figure 2. Layered architecture and component relationships. View the figure in the GitHub repository

4.2 Object-oriented design and patterns

An abstract User class supports polymorphic role objects for Administrator, Lecturer, Student and HoD. A factory creates role instances from persisted role values. A Strategy represents late-penalty policy so the calculation can be tested independently and changed without rewriting the marking service. Repository/data-access abstractions isolate persistence. These patterns support single responsibility, dependency inversion and controlled variation while avoiding unnecessary framework complexity.

The design also models ownership relationships: lecturers access only assigned courses, HoDs access only their department, and students access only their own enrolled records. The design makes these scopes explicit because role labels alone are insufficient security controls.

Figure 3. Class, activity, state and sequence diagrams for the assessment lifecycle. View the figure in the GitHub repository

4.3 Behavioural models

The submission lifecycle moves through submission, late determination, marking, moderation and release. An activity flow represents the student submission path, including validation and version creation. A moderation sequence represents lecturer marking followed by HoD review, rejection or approval, and release only after all entries in the selected batch are approved. These models guided both service implementation and acceptance tests.

4.4 Database design

The schema is normalised around user accounts, role profiles, departments, courses, course-lecturer links, enrolments, assignments, extensions, submissions, submission versions, marks, moderation decisions and audit records. Foreign keys enforce relationships and indexes support common lookups. The Phase 8 change adds a class-wide extension relation rather than copying a deadline into every student record. This preserves a single source for the class policy while allowing individual extensions to take precedence.

Figure 4. Entity-relationship/database design and schema mapping. View the figure in the GitHub repository

Design evidence: Phase 3 design document and UML diagrams, database schema

5. Implementation

5.1 Application structure and workflows

The implementation is organised into headers, source files, services, data-access code, console UI and tests. The Administrator dashboard manages accounts, departments, courses, lecturer assignments and student enrolments. The Lecturer dashboard manages assignments, extensions, submissions, marks and feedback. The HoD dashboard opens assessment batches, reviews marks, approves or rejects decisions and releases approved results. The Student dashboard shows enrolled courses, active assignments, effective deadlines, submission status and released results.

5.2 Persistence and transactions

All persistent operations use SQLite through the data-access layer and parameterised statements. Transactions group logically inseparable changes. For example, creating or changing a mark and writing its audit entry must succeed together. If the audit write fails, the mark and related status change are rolled back. Foreign-key checks and validation prevent partial or inconsistent records.

5.3 Security

Passwords are never stored in plaintext. The application uses OpenSSL PBKDF2-HMAC-SHA256 with a unique salt and an iteration count. Login derives a verifier from the entered password and compares it with the stored value. The console disables terminal echo during password entry. Role and record-scope checks deny unauthorised reads and writes. Parameterised queries reduce SQL injection risk, while audit records provide accountability for mark changes.

5.4 Assessment rules

A submission stores the student, assignment, version, receipt time, file metadata and checksum. Effective deadlines consider the original deadline, an individual extension and a whole-class extension, with individual precedence. Late days use the defined ceiling rule. The deduction is 10% of the maximum mark per late day for up to three days, and the result cannot fall below zero. HoD release is blocked until every mark in the selected assessment batch is approved.

6. Verification and validation

6.1 Test strategy

The V-Model maps requirements to unit, integration, system and acceptance testing. GoogleTest covers service-level behaviour and database interactions. Boundary values include marks of zero and maximum, exactly-on-time submissions, one-minute late submissions, 24-hour and 72-hour boundaries, and a submission one second beyond 72 hours. Negative tests cover invalid roles, wrong course scope, non-enrolled students, missing files, malformed hashes and failed audit writes.

6.2 Results and coverage

Measure	**Verified result**
Automated regression suite	39 of 39 tests passed; 0 failed.
Business-logic line coverage	80.7% (774 of 959 lines).

All-source line coverage	59.8%, including interactive console and entry-point code.
Business-logic function coverage	94.6% (70 of 74 functions).
Business-logic branch coverage	38.3% (861 of 2,250 branches).
Acceptance scenarios	Administrator setup, submission, resubmission, marking/audit, moderation/release and student result visibility.

The two metrics are reported separately for transparency. The 80.7% measure excludes the interactive UI and main entry point because those paths are tested through manual acceptance; the 59.8% value includes all production source. The project therefore exceeds the assignment's 70% business-logic coverage target without hiding the lower automated coverage of console interaction.

6.3 Validation

Manual validation used synthetic accounts and data to verify the end-to-end workflow. The evidence confirms that students cannot see marks before release, lecturers can mark within their course scope, HoDs cannot release incomplete batches, and audit records include actor, action, values, reason and timestamp. The Phase 8 acceptance scenario additionally confirms that the whole-class extension is displayed to enrolled students and used for on-time/late calculation.

Figure 5. Automated test, coverage, audit-log and acceptance evidence. View the figure in the GitHub repository

Verification evidence: Phase 5 test results, Phase 6 evidence document, Phase 8 regression results

7. Configuration management and quality assurance

Git is the configuration-management system. The repository contains the source, schema, tests, build guide, traceability matrix and phase evidence. Build products, databases and secrets are excluded. Commits describe implementation and evidence changes. The CI workflow builds the project, runs CTest and performs static analysis on pushes and pull requests.

Cppcheck inspected the production source files with warnings, style, performance and portability checks enabled. The release quality gate requires a successful C++17 build, passing tests, no critical open defects, static-analysis review and completed evidence. The defect log records the password-entry correction, database/evidence issues and their retest status. The quality record distinguishes automated metrics from manual console validation, which improves rather than weakens the credibility of the submission.

QA evidence: Phase 6 QA report, defect log

8. Deployment

The deployment target is Windows with WSL2/Ubuntu or a comparable Linux environment. A technician installs the compiler, CMake, Ninja, SQLite development files, OpenSSL development files and GoogleTest dependencies, then configures and builds the project. The application creates or opens a local SQLite database using the supplied schema. Database backups are file copies made while the application is stopped or after a verified safe shutdown.

The installation guide explains build commands, database initialisation, first administrator setup, role creation and safe shutdown. The user manual documents each dashboard and example workflows. The deployment diagram shows the terminal and LMS executable, SQLite database, schema, OpenSSL security library and development-only testing/static-analysis tools.

Figure 6. Deployment diagram for the local WSL2/Linux installation. View the figure in the GitHub repository

Deployment evidence: installation guide, user manual, release notes

9. Maintenance: change request CR-01

After version 1.0, the school requested that a lecturer be able to extend the deadline for a whole class rather than only for an individual student, and that the extension be visible in the student assignment list. This is perfective maintenance because it improves a valid operational capability without changing the fundamental assessment lifecycle.

The impact analysis identified the schema, assignment service, student service, console menus, deadline calculation, tests and release documentation. A persistent class-extension table stores the assignment, later deadline, reason, granting lecturer and timestamp. The Lecturer must be assigned to the course. The Student service displays the class extension label and uses the new deadline. An individual extension remains more specific and therefore takes precedence.

Version 1.1 added two regression tests. The complete suite passes 39/39, including the new test proving that students see and use the class-wide deadline for late calculation. The change was accepted because it satisfies the acceptance criteria and preserves all earlier behaviour.

Figure 7. CR-01 whole-class extension and 39/39 regression evidence. View the figure in the GitHub repository

Maintenance evidence: change request, regression results, v1.1 release notes

10. Professional, legal and ethical issues

The system processes usernames, submissions, marks, feedback and audit history. The school must define a lawful purpose, retention period, access responsibilities and backup policy before real learner data is used. The Data Protection Act 2012 of Lesotho is a relevant legal source; the project does not claim that technical controls alone complete the school's compliance obligations. Child data, data-subject rights, security incidents and deletion/retention require institutional decisions and appropriate notices.

Ethically, the late-penalty rule must be visible and consistently applied. The extension mechanism supports legitimate circumstances but must be recorded with a reason and authorised by the correct Lecturer. Moderation reduces the risk of an individual mark being released without review. The audit log supports accountability but should itself be protected from unauthorised modification. The simple console, clear messages and role-specific manuals support accessibility for users with limited technical experience.

Professional integrity also requires transparent reporting. The project reports both the business-logic coverage target and the lower all-source figure, describes the limitations of manual UI testing, and excludes unsupported claims about concurrent users. This is preferable to presenting one favourable metric without its scope.

11. Conclusion

The LMS demonstrates a complete software-engineering lifecycle. Stakeholder conflicts were converted into measurable requirements; the layered object-oriented design maps to a normalised SQLite schema; implementation provides four scoped roles, secure password storage, persistent assessment workflows and auditability; V-Model verification demonstrates 39 passing tests and 80.7% business-logic line coverage; QA establishes configuration and static-analysis controls; deployment documentation supports WSL2/Linux installation; and CR-01 demonstrates controlled post-release maintenance.

The remaining limitations are appropriate to the approved scope. The application is a console system rather than a web/mobile LMS, the interactive UI is primarily validated through acceptance walkthroughs, and concurrent network use is not claimed. Future work could add a graphical interface, schema migration tooling, broader automated UI testing and a service architecture for multi-user deployment. Within the assignment scope, the released system is traceable, testable and suitable for demonstration.

12. References

CMake Project. (n.d.). CMake documentation. [online link]
GoogleTest. (n.d.). GoogleTest primer. [online link]
Lesotho. (2012). Data Protection Act, 2012 (Act No. 5 of 2012). Lesotho Legal Information Institute. [online link]
National University of Lesotho. (2026). CS4433 software engineering individual assignment [Assignment brief]. [online link]
OpenSSL Project. (n.d.). OpenSSL documentation. [online link]
SQLite. (n.d.). SQLite documentation. [online link]
LMS project repository. (2026). CS4433-LMS-Phase4 [Source code and evidence repository]. [online link]

13. Appendices

Appendix A: Requirements traceability

The complete traceability matrix links each functional and non-functional requirement to its use case, design element, database table, source file and test case. It is maintained as a CSV file in the repository so that the complete matrix remains searchable without expanding the main report.

Open Appendix A: requirements-traceability.csv

Appendix B: Full test and QA evidence

The full test results, coverage commands, static-analysis evidence, defect log and screenshot placement records are retained in the repository. The linked documents are the authoritative evidence package for the viva and assessment review.

Open the evidence folders: Phase 5 evidence | Phase 6 evidence | Phase 8 evidence

Appendix C: User manuals and deployment records

The deployment guide, user manual, release notes and deployment diagram provide the technician and user-facing material required for installation and demonstration. These are linked rather than copied into the report to avoid repeating operational instructions.

Open deployment records: Phase 7 documents